Route Classification in Bouldering Gyms

Eric Buys & Myron Ladyjenko

University of Guelph
School of Computer Science
November 29, 2024

Individual Contributions

Hi, my name is Myron. I worked on setting up the YoLo and Mask R-CNN(detectron2) models for our project. I was able to get the two models working and together with the obtained dataset train them for object (climbing holds) detection. On my laptop, I tried training them using CPU, but after I realized that Eric had a GPU and we could leverage that, which is what we set up and trained our models with. Another thing I worked on was converting the Mask R-CNN bounding box format and labels (i.e. predictions) to the YoLoV11 format as Eric set up the K-means algorithm to work with YoLov11 formatted data.

My name is Eric and I worked on a few key parts of our project. While Myron tackled more of the object detection and setup, I decided to work on clustering with detected holds. The first of these was using the K-Means algorithm for intra-detection clustering. In addition to this, for inter-detection clustering between different climbing holds, the DBSCAN algorithm was used. The last key part I worked on was setting up an additional computer for training our YoLo/Mask-R-CNN models on a GPU.

Introduction

In this project, we explore how to create a system for classifying climbing routes from an image. Throughout this project, we explore a variety of Machine Learning-related models such as Mask-R-CNN, YoLo_v11, K-means and DBSCAN. Upon building this system, the effectiveness of our system has been analyzed and various areas of improvement are proposed throughout. We focus on exploring different approaches and then converging on one to pick as our solution to a problem.

Motivation

Outside of focusing on our studies, both of us are avid climbers and have been climbing since the beginning of our second year, almost 3 years now. One day when we went climbing, we met a guy who looked like he was having a hard time figuring out which holds to use. Even when he began climbing, he looked back and asked us if the foothold he was pointing to was his to use or not. This seemed odd to us so we asked him about it and he told us he was colour blind, and it was hard to distinguish between certain holds (eg: red and green, orange and blue, etc). Not too long after, we participated in a hackathon and we decided to build an app for climbers to keep track of their completed climbs, where it would also have the ability to outline holds for colour-blind users. At the time, we had minimal

experience in machine learning, so our approach to outlining holds was very rudimentary and involved manually highlighting holds for a climb.

Now, when we got the opportunity to build anything we wanted for CIS*4780, we thought this was the perfect opportunity to combine a sport we love, improve upon a past project of ours, and apply some computational intelligence techniques.

Literature Review

As was mentioned above, we both like rock climbing. What might have been omitted is that we also very much enjoy computer science, software engineering and specifically computational intelligence. As we explored the World of rock climbing we were thinking about different problems to solve. We explored some of the existing projects done by other people in the space of Machine Learning and Rock Climbing.

The paper *'Indoor Rock Climbing Wall Route Displayer'* [5] gives insight into climbers' positions on the wall, and because of that, it allows route setters to enhance the experience of climbers. Part of the paper also addresses the problem of detecting holds from an image and creating new routes for someone to climb, which is what our primary focus will be.

While considering different approaches to creating a model for our application, we considered different models such as YoLo, Mask R-CNN, DeepLab, RESNET and SAM2. We found a resource 'https://github.com/tonylay7/bouldering_route_classification' [8] which contains implementation for a Mask R-CNN model for classifying routes, which helped us to find a starting point for working with this model and gave some suggestions that we could try in our experiments.

As a future improvement, our ultimate goal is to detect a climber on the wall from a submitted video. The paper '*Automatic Sensor-Based Detection and Classification of Climbing Activities*' [1] provided us with some insights into how analysis is done on the video to determine which type of movement is occurring, which will be beneficial for our goal. Another paper that was quite useful for us was *'Computer Vision-Based Indoor Rock Climbing Analysis'* [3] as it focuses on the analysis of valid holds and the number of moves.

Problem Statement

Climbing gyms typically rely on colour-coded holds to define routes. This makes it tough for colour-blind people to differentiate which holds they can and can't use for a route. We sought to address the issue by creating an easy tool to aid colour-blind climbers in identifying holds on the wall.

Dataset

We managed to find a few datasets online used in similar rock-climbing projects. The one we settled on was a dataset from roboflow[2]. This dataset included 4306 labelled images, already with augmentations applied as well. In addition to this, the data was split into 3 groups, a training set (3876 images), a test set (148 images) and a validation set (277 images). The dataset included All of this data was already accessible in both YOLOv11 and COCO formats. In addition to this, since we did not construct this massive dataset from scratch, we went out and collected some extra images from the local climbing gym here in Guelph ('The Guelph Grotto'). The dataset labelled holds in the images into two categories: 'holds' and 'volumes'. Lastly, a subset of this big dataset was used to train models quicker, which consisted of 240 images in the training set.

To construct a custom dataset, we used the Roboflow framework. The framework allows for uploading custom images and using the annotation tool, we could draw bounding boxes and label the holds (i.e. 'regular' vs 'volume' hold, etc.). Once all of the images were annotated, the platform allowed us to extract the data in the appropriate format based on the model we were using. The two formats that we used were YOLOv11 and COCO for the YOLO and Mark R-CNN models respectively. The difference lies in how the bounding boxes are represented for the respective hold in the images. Both of the datasets have images and YOLO formats have labels, which represent normalized coordinates of the bounding box and the label that is assigned to each box: <class_id> <x_center> <y_center> <width> <height>. Similarly, the COCO format has images, but it uses annotation files to store image metadata (image ID, file name, height, width) and bounding box dimensions: "[x_min, y_min, width, height] for each bounding box with the label. Note that one format can be easily converted into another one which we will leverage to cluster holds of the same color to identify a route.

Design & Methodology

Once the dataset was all set up, we could start working on the actual models for our project. The first is the object detection model. Here we had two choices, either the YoLov11[6,7] or the Mask-R-CNN[4] model. Both of these models took a long time to train, especially on our laptops which forced us to break out a desktop with a GPU for training. Once we did this, our training speeds increased ~4x and we reduced training times for the YoLo model from ~1hr to ~15 min/epoch. Additionally, the model was saved after each epoch so we could assess the status of the object detection model over time.

Next, we tried a different model Mask R-CNN (detectron2). We used it for object detection - identifying climbing holds. We used the same dataset as for the YoLoV11, the

only difference is in the format - COCO. During the setup, we encountered compatibility issues with the torch, cuda and detectron2 package versions, but we successfully resolved them. During the initializing and training phases of the model, we noticed that this model has more configurations that are needed to train it compared to YoLo. This could be a benefit as it allows for more flexibility but requires a deeper understanding of the internal workings and different parameters. Mask R-CNN took ~x2 time to train for one epoch on the same dataset and the model takes significantly more space - ~60x times. Overall, this model allowed more precious but required more time for training. Another advantage of using Mask R-CNN is that its primary usage is instance segmentation which can be very beneficial in our further investigations as we continue to work on this project.

The second model we worked on is a K-Means algorithm for determining the hold colour for a given detection. Since we know that an object detection will contain a hold which is centrally located in a small subset of an image, the majority of the image will either be the hold itself, or parts of the climbing wall behind it. This can be seen in the figure below.

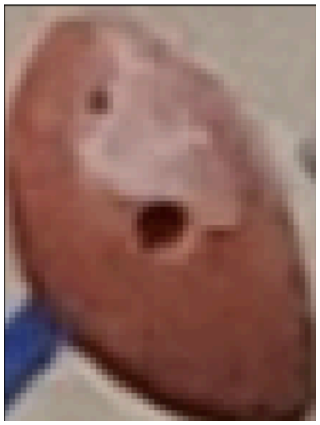


Figure 1: Orange Climbing Hold Detection

Using this assumption, it is a safe bet to begin with k=2, as there will only ever really be two main colours in the image, the background colour of the wall and the colour of the climbing hold. Since all the holds will typically share the same background colour, we can set a mask equal to the background colour of the climbing wall. Using this value, we can compute a dissimilarity measure to figure out which one of the two colours from the K-Means algorithm is most different from the climbing wall colour to determine the colour of the hold in that detection.

An enhancement that can be made to the described above K-Means algorithm is to apply Gaussian Blur to the image before the algorithm. The Gaussian Blur method is used to reduce noise in the image (smooth the image) by averaging pixel values based on their neighbours using a kernel matrix. This algorithm can help remove noise or subtle

variations in colour/shading, which could make it easier for K-means to identify distinct colours of the hold more effectively. Further, we could try GMM (Gaussian Mixture Model), which uses probabilities that get assigned to each data point, which could allow us to handle cases where climbing holds might not be 'fully' one colour (i.e. red hold with black marks from shoes and white from chalk).

The last model we used was the Density-Based Spatial Clustering of Applications with Noise model (DBSCAN). This model is great at finding different clusters of data without needing to be told the number of clusters in the data. Due to this, it is a prime option for applying to the problem of inter-detection clustering. This is because we want to group all the hold detections that have similar colours.

Results

After various epochs of the YoLo model, we achieved some relatively decent performance. From the diagrams below, you can see 3 different YoLo models running on the same image, attempting to identify all the different holds in the image. If you look closely at the three different diagrams, as the model is trained more, it is able to get a higher confidence for each hold detection and also get a higher quantity of correct hold detections. One limitation with the current YoLo model however is that it struggles to distinguish actual climbing holds from tape on the wall to indicate the start/end of a climb. Due to the lack of such obvious tape in the dataset, the model is not exactly trained on the fact that tape is not the same as a hold, and thus it has a hard time determining that tape is not actually a climbing hold.

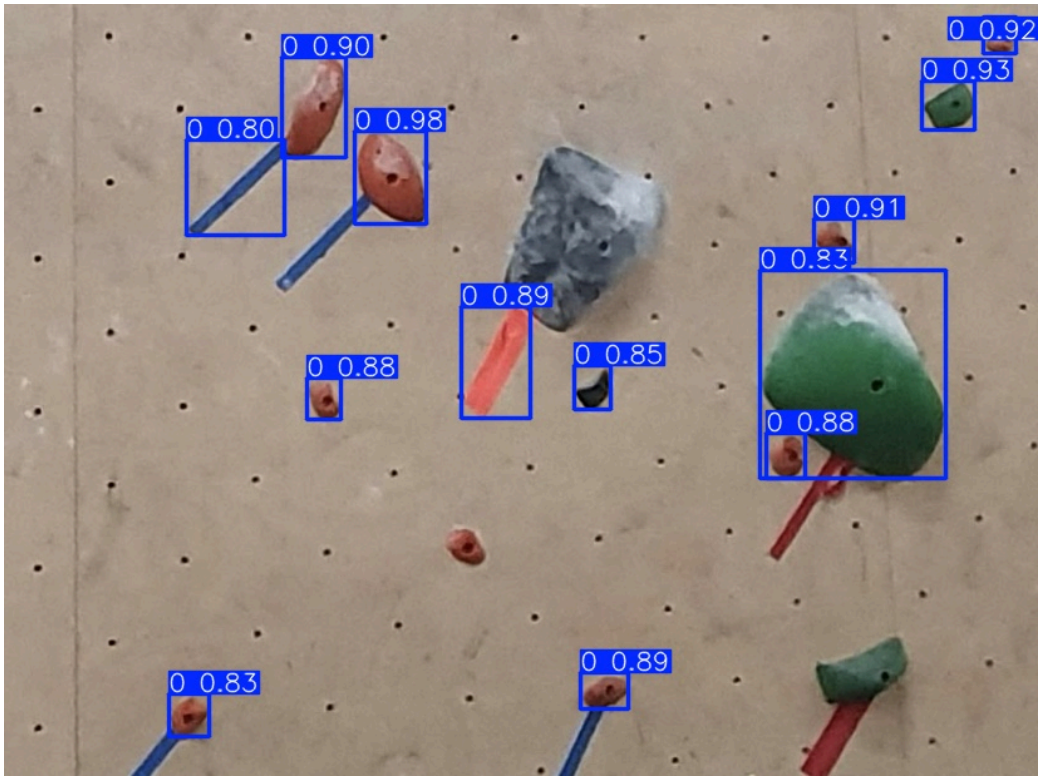


Diagram 1: YoLo Model - Epoch #2 (240 images/epoch)

Diagram 2: YoLo Model - Epoch #5 (4000 images/epoch)

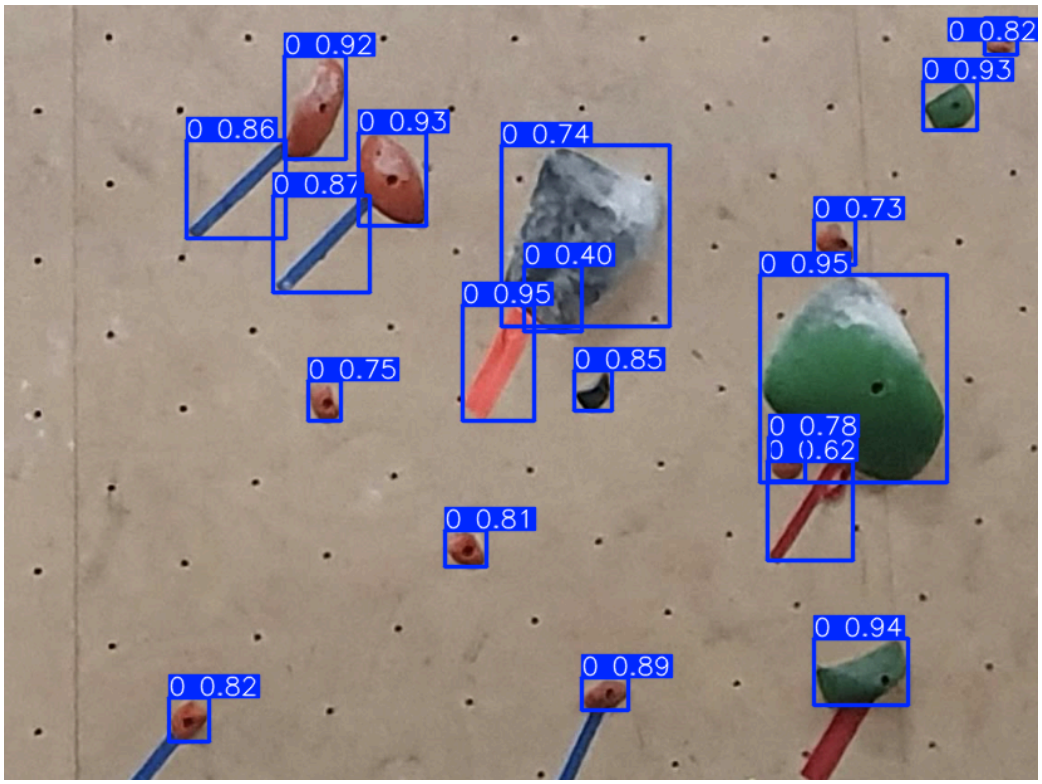
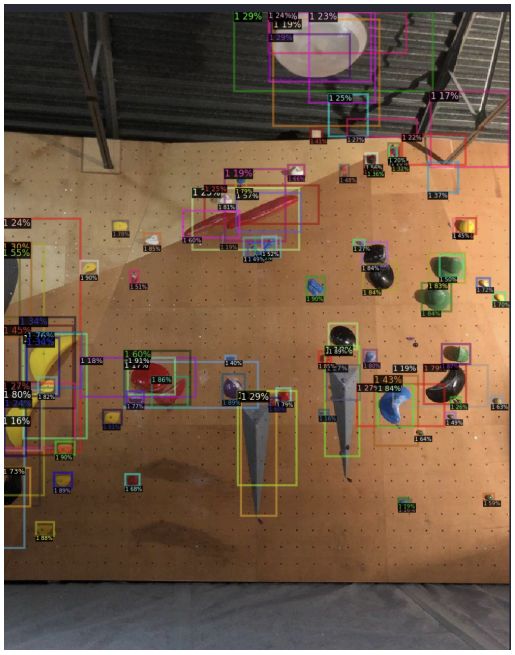
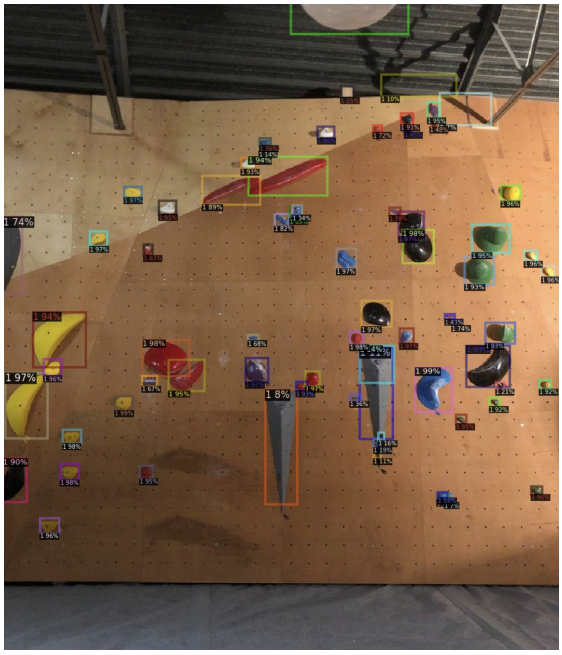


Diagram 3: YoLo Model - Epoch #15 (4000 images/epoch)

Next, for object detection, we used the Mask R-CNN model and tried to compare its results to the results of the YoLo model. First, let's see how the training affects the performance of the model. As we can see, in the first photo, the model detected a lot of objects that are not holds. This is due to the limited exposure to the data and very limited training. On the contrary, even when we do train the model more extensively, we get much better results, as can be seen in the photo on the right.





Diagrams (a), (b): (a) - 100 iterations (200 images), (b) - 2000 iterations (4000 images - ~1 epoch)

We saw that Mask R-CNN performed better by detecting tape on the wall less frequently when compared to the YoLo version:

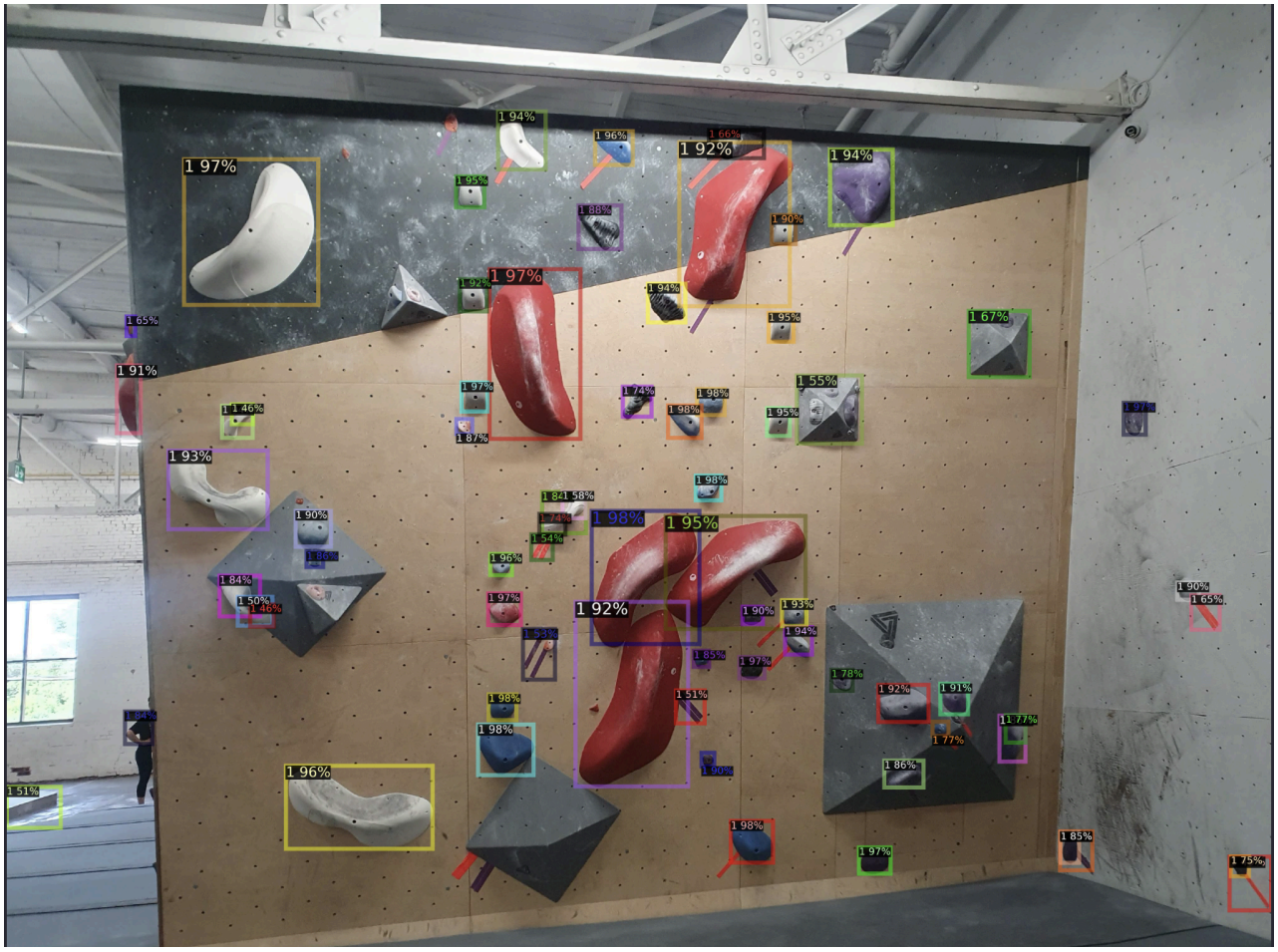
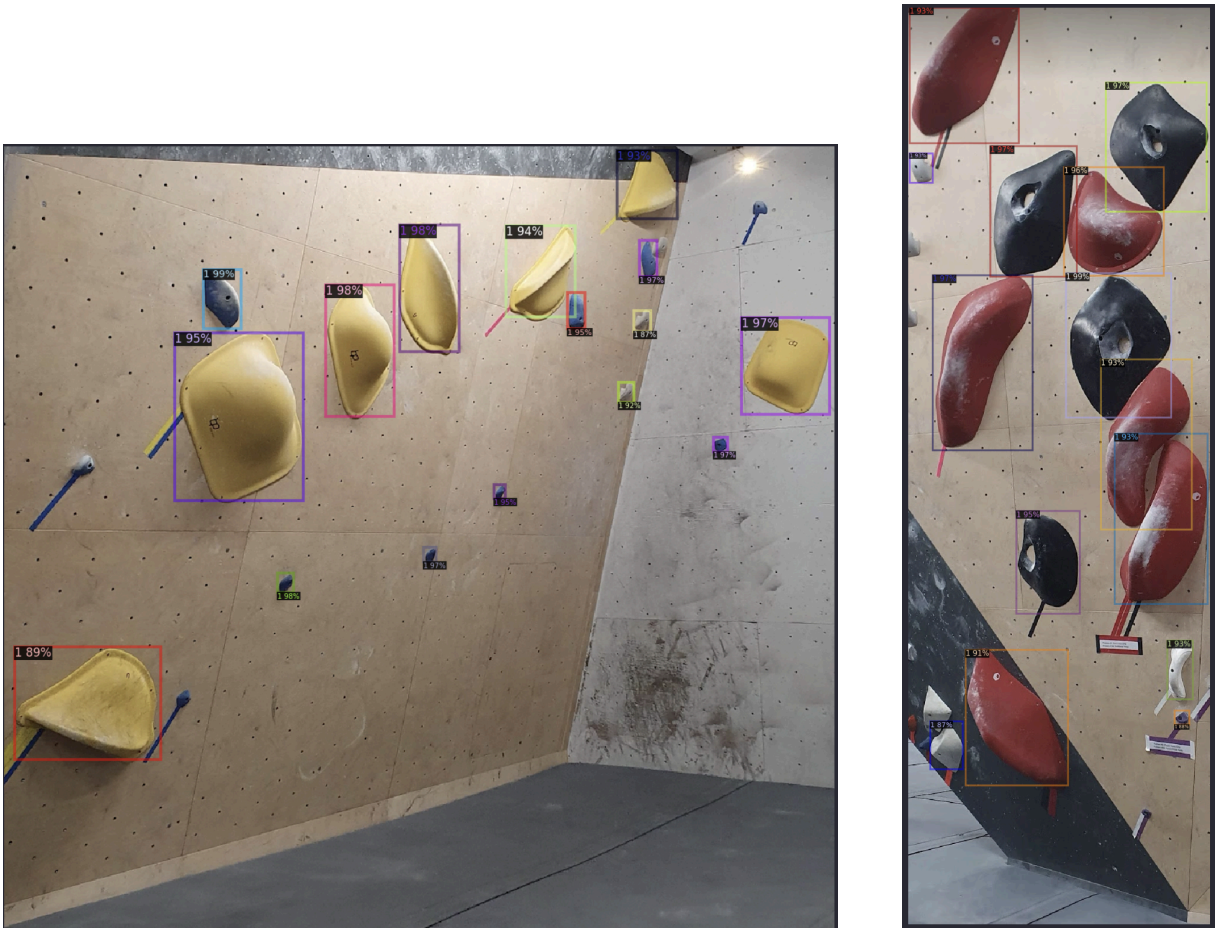


Diagram (c): very few tapes detected with a high accuracy of detecting holds.

After training the model, we performed additional tests on photos we took at 'The Guelph Grotto' climbing gym. We can see that in both photos, the model does a very good job of detecting the holds. Even though we currently use the Mask R-CNN model to detect objects inside of a bounding box, we believe that utilizing segmentation will offer even more improvements. This is because precise segmentations of individual holds can give the ability to handle scenarios where holds vary in shape, size, and colour and be closely positioned. Some more example predictions from our model can be seen below:



Diagrams (c), (d) - clear detection of the holds, no tape was detected (some holds were still missed)

Once the KMeans Algorithm was implemented, we ran it on the hold detections from our YoLo model. From diagrams 4, 5 and 6, you can see the result of our algorithm.

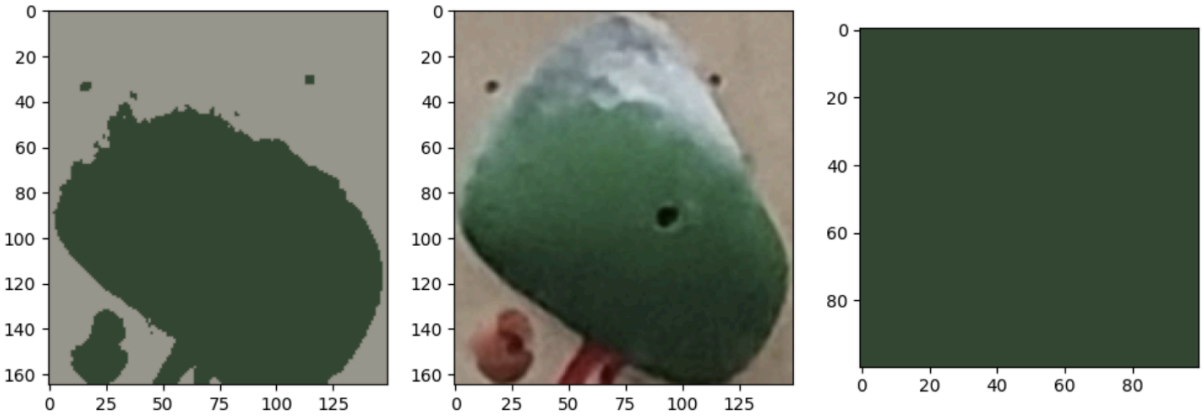


Diagram 4: K-means Model plot, Original Green Hold Image, Selected Colour

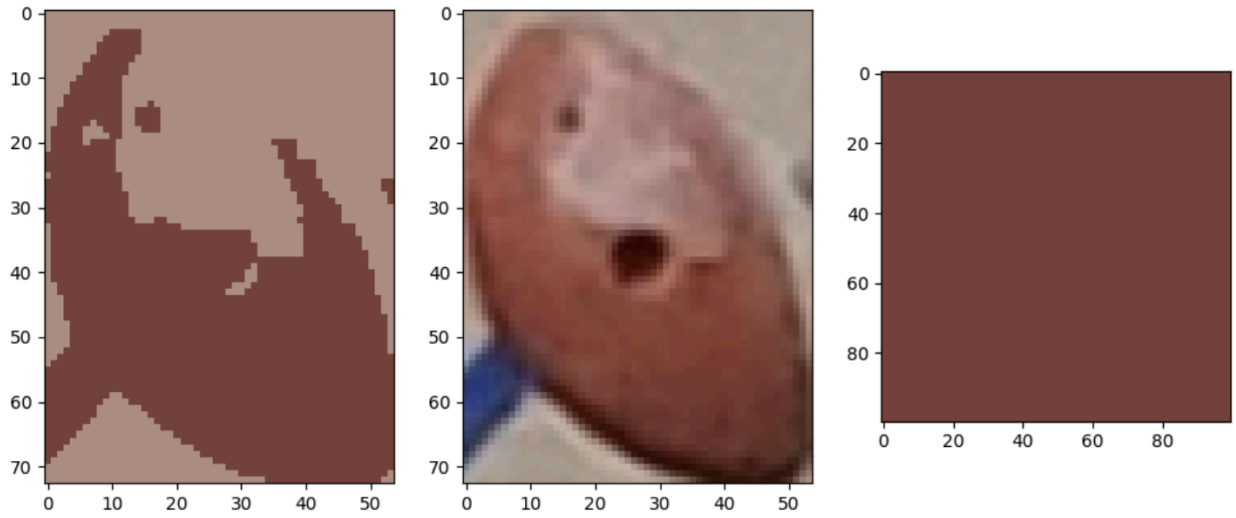


Diagram 5: K-means Model plot, Original Orange Hold Image, Selected Colour

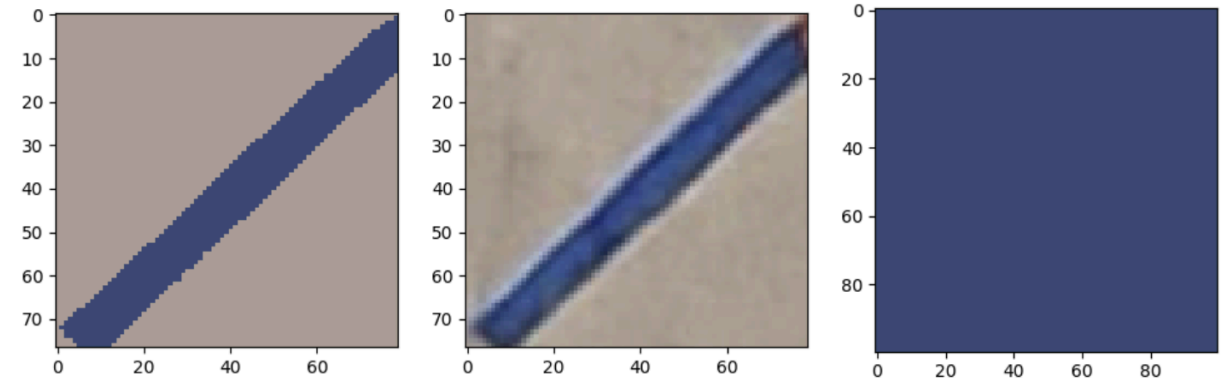


Diagram 6: K-means Model plot, Original Blue Tape Image, Selected Colour

In cases where the object was vastly different from the background wall colour, it was very easy to determine which colour to use. This is because, after the k-means algorithm, two colours were chosen. From these two colours, a calculation comparing the norm of each colour vector to the wall colour was calculated to determine which hold to use. This calculation can be seen in Code Snippet #1.

```
if np.linalg.norm(color1 - background_colour) > np.linalg.norm(color2 - background_colour):
    return color1
else:
    return color2
```

Code Snippet #1

However, one small issue that results from the object detection model is that if an invalid object is passed, like the one containing blue tape in Diagram 5, we still carry on determining the colour. This is because at this point in our pipeline, it is assumed that all object detections are valid, and as such our K-means model will run without issue on objects that aren't climbing holds, especially since the tape is still bright-coloured.

The last part of our pipeline is to group these detections based on the selected colours from the K-means algorithm. After taking a look at diagrams 7, 8 and 9 below, we can see that there is some success and some room for improvement using this technique. In diagram 9, it appears to have grouped all blue detections perfectly as they are nicely grouped together. When looking at diagram 8, it has grouped most of the orange holds together, and because a red tape was passed along, it also grouped a red tape along with the rest of them. This was likely due to the fact that minPoints in DBSCAN was set to 2, and thus the red tape couldn't belong to its own cluster. Similarly, in diagram 7, we can see it clustered the green holds together, plus a grayish detection.

Overall, from the DBSCAN algorithm, we can see that it has some successes when clustering holds that are very similar in colour. However, when there is extra "bad" data being passed to it, it needs to find a place to put it and lump it with the closest colour clustering available.

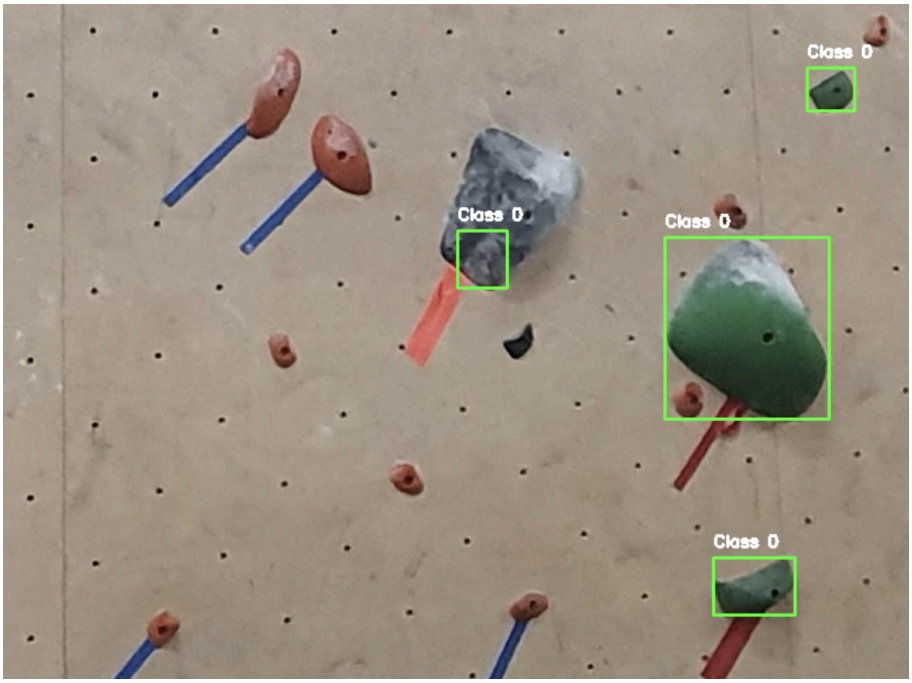


Diagram 7: Grouped Holds by Green Colour

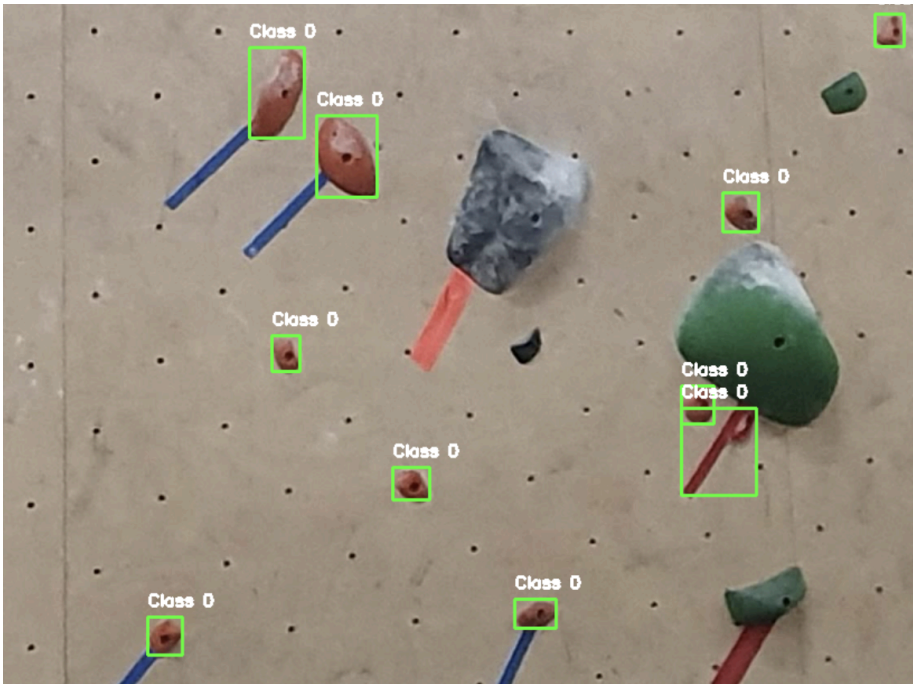


Diagram 8: Grouped Holds by Orange Colour

Diagram 9: Grouped Holds by Blue Colour

Future Work

There are two main directions for future work on this project as of now. The first is short-term improvements related to overall algorithmic/model improvements. This includes improving the object detection models, specifically, so that the object detection model will detect all holds and volumes. Additionally, it will need to be improved to stop detecting tape on the wall as well as some graphics on the walls of some gyms. Another direction this can be taken is using segmentation. Mask r-CNN supports segmentation and that would be a logical next step as it would have great benefits for the K-means algorithm in the next stage of our data processing pipeline.

Beyond the object detention model, the model we use to detect the colour of a hold still has room for improvement. It lacks the ability to deal with holds that have chalk on them and struggles with bad lighting as well. One way to do this is by using the Gaussian blur method mentioned above. The last room for improvement in our models is related to the DBSCAN model. As was mentioned during the presentation, the SigLIP model may be beneficial to assist in this stage.

Beyond just algorithmic improvements, the next step for this project once the route classification has been completed properly, is being able to verify if a person has completed a climb from a video. Using this model will need to be expanded to verify if a person properly starts a climb, only uses holds that are actually part of the climb throughout the video, and properly finishes the climb.

Conclusion

Throughout this project, we had the opportunity to develop a system for detecting climbing holds in an image, pass those detected holds to a colour clustering algorithm to determine the colour of the hold and finally pass those colours to another clustering algorithm to group the colours together in their respective groups. This allowed us to end up with a system that can take a picture of a climbing wall and classify the routes of a given image. Additionally, by comparing the YoLo and Mask-R-CNN models, we were able to have choices between two very good models, as well as easy adaptations for future work as Mask-R-CNN is suited towards image segmentation as well.

References

1. Automatic sensor-based detection and classification of Climbing Activities | IEEE Journals & Magazine | IEEE Xplore. (n.d.-a). https://ieeexplore.ieee.org/document/7274672/
2. Blackcreed. (2024, June 11). *Climbing holds and volumes object detection dataset by blackcreed*. Roboflow. https://universe.roboflow.com/blackcreed-xpgxh/climbing-holds-and-volumes
3. Computer Vision based indoor rock climbing analysis. (n.d.-b). https://kastner.ucsd.edu/ryan/wp-content/uploads/sites/5/2022/06/admin/rock-climbing-coach.pdf
4. He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2018, January 24). *Mask R-CNN*. arXiv.org. https://arxiv.org/abs/1703.06870
5. Indoor Rock Climbing Wall route displayer. (n.d.-c). https://stacks.stanford.edu/file/druid:bf950qp8995/Wei.pdf
6. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016, May 9). *You only look once: Unified, real-time object detection*. arXiv.org. https://arxiv.org/abs/1506.02640
7. Author links open overlay panelPeiyuan Jiang, & AbstractObject detection techniques are the foundation for the artificial intelligence field. This research paper gives a brief overview of the You Only Look Once (YOLO) algorithm and its subsequent advanced versions. Through the analysis. (2022, February 3). *A review of Yolo algorithm developments*. Procedia Computer Science. https://www.sciencedirect.com/science/article/pii/S1877050922001363

8. tonylay7. (n.d.). *Tonylay7/bouldering_route_classification: A third year project that revolves around classification of climbing routes using image processing and deep learning techniques*. GitHub. https://github.com/tonylay7/bouldering_route_classification